THREAT MODEL

Flask Blog Application

CYC386 — Secure Software Design and Development

Team Name	Flask Blog Security Team
Members	Esha Khan [SP23-BCT-014] | Husna Anjum [SP23-BCT-020] | Zainab Shakil [SP23-BCT-052] | Zarmeena Khan [SP23-BCT-053]
Date	April 2026
Course	CYC386 Secure Software Design and Development
Instructor	Engr. Muhammad Ahmad Nawaz

1. Introduction

This Threat Model document was produced as part of the CYC386 48-Hour DevSecOps Security Sprint. It covers the Flask Blog Application — a multi-user blogging platform built with Python Flask, SQLAlchemy, Flask-Login, and Flask-WTF.

The application allows users to register, log in, create posts, and manage their own content. This document systematically identifies threats using the STRIDE methodology, assesses risk using CVSS v3.1, and defines mitigations for each threat. Version 1.1 updates the STRIDE table and Mitigation Summary to reflect the completed security implementations from Sprint Phase 3.

1.1 Application Overview

- Framework: Python Flask (Blueprint architecture)
- Database: SQLite via SQLAlchemy ORM
- Authentication: Flask-Login with bcrypt password hashing
- Forms: Flask-WTF (WTForms with CSRF support) — CSRF protection ENABLED
- Repo: github.com/EshaKhan-ek/SecureApp-Sprint-Flask-Blog-App
- Base Folder: Python/Flask_Blog/11-Blueprints

1.2 Scope

- User registration and authentication (login/logout)
- Blog post creation, editing, and deletion (IDOR fix applied)
- User profile management (ownership check applied)
- Session and cookie management (SameSite=Strict, HttpOnly, Secure flags)
- CSRF protection (Flask-WTF globally enabled)
- Clickjacking protection (X-Frame-Options + CSP headers)
- Error handling and information disclosure (debug=False enforced)

2. Data Flow Diagrams (DFDs)

Data Flow Diagrams show how data moves through the Flask Blog Application. Trust boundaries are marked with dashed lines to indicate where data crosses security zones.

2.1 DFD Level 0 — Context Diagram

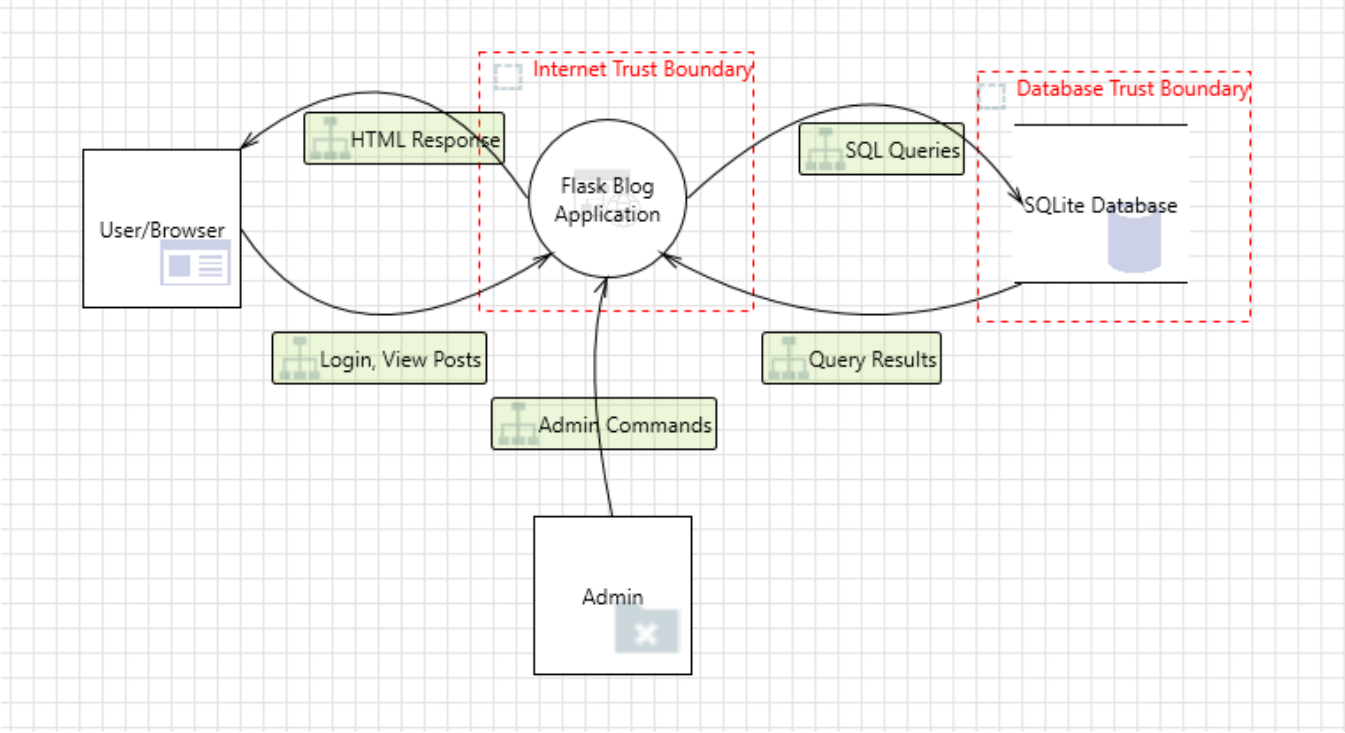

Figure 1: DFD Level 0 — System Context Diagram

Data Flows:

- User/Browser → Flask Blog App: Login request, registration, view/create/edit/delete posts
- Flask Blog App → User/Browser: HTML pages, session cookies, CSRF tokens, error responses
- Admin → Flask Blog App: Admin commands, user management
- Flask Blog App → Database: SQL queries via SQLAlchemy ORM (SELECT, INSERT, UPDATE, DELETE)
- Database → Flask Blog App: Query results, user records, post data, session tokens

2.2 DFD Level 1 — Decomposed

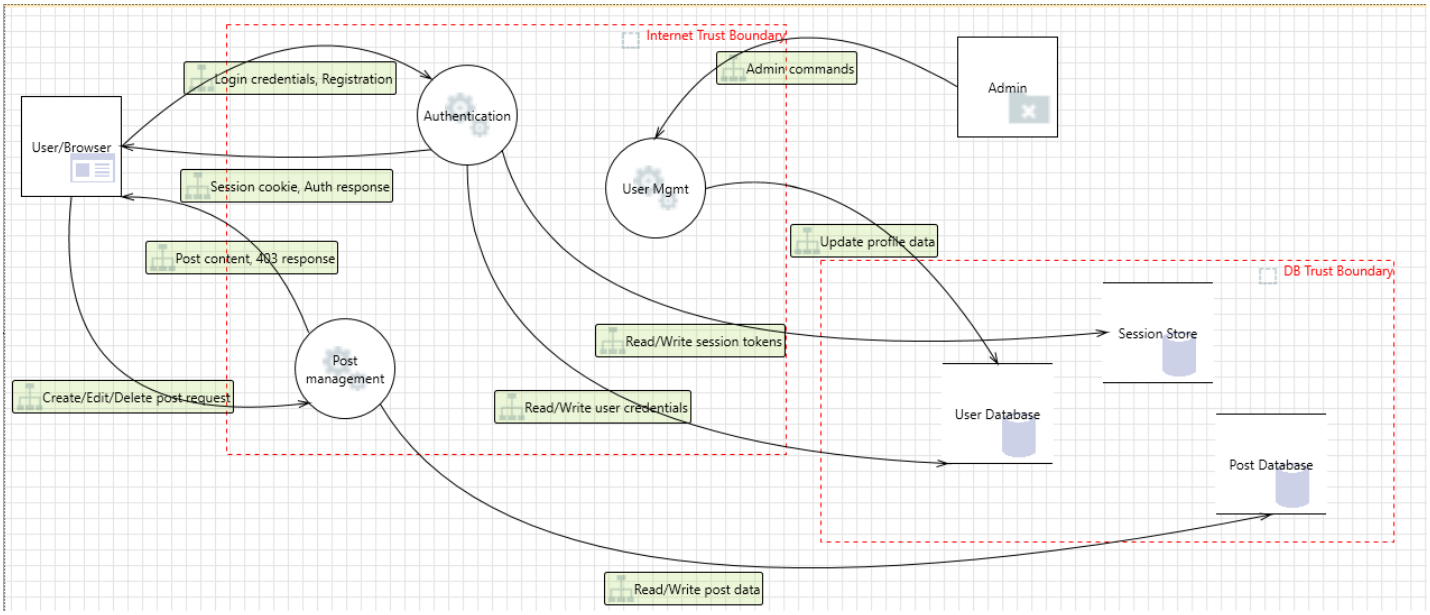

Figure 2: DFD Level 1 — Decomposed with Trust Boundaries

Processes:

- P1 — Authentication: Handles login, logout, registration, password hashing (bcrypt)
- P2 — Post Management: CRUD operations on blog posts (IDOR fix applied — abort(403) ownership check)
- P3 — User Management: Profile updates, account settings, avatar upload (ownership check applied)
- P4 — Comment System: Comment creation and moderation

Data Stores:

- DS1 — User DB: Stores user credentials, profiles, bcrypt-hashed passwords
- DS2 — Posts DB: Stores blog posts with author foreign key (IDOR-protected)
- DS3 — Session Store: Flask session data, CSRF tokens (SameSite=Strict, HttpOnly, Secure)

Trust Boundaries:

- Internet Trust Boundary: Separates external users from internal application processes
- DB Trust Boundary: Separates application processes from data storage layer

3. STRIDE Threat Analysis

STRIDE is a threat modeling methodology developed by Microsoft. Each letter represents a category of threat: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. The table below has been updated to reflect the security fixes implemented in Sprint Phase 3.

Status legend: ✓ FIXED = implemented and verified | ✓ Done = pre-existing control | Pending = scheduled for implementation

ID	STRIDE	Component	Threat Description	Mitigation	Status
T-01	Spoofing	Login endpoint	Attacker impersonates valid user using leaked or brute-forced credentials	Bcrypt password hashing (Flask-Bcrypt) + Flask-Limiter rate limiting on /login (10 req/min per IP)	**✓ Done**
T-02	Tampering (IDOR)	POST /post/<id>/update	IDOR: attacker changes post_id in URL to edit another user's post without authorization	Ownership check: if post.author != current_user: abort(403) — applied to update_post() and delete_post()	**✓ FIXED — Member 1**

T-03	Repudiation	Blog CRUD actions	No audit trail — users can deny performing create/edit/delete actions	Server-side logging of all CRUD events with user ID, timestamp, and action type	**Pending**
T-04	Info Disclosure	Error pages / run.py	Flask debug=True exposes full stack traces and Werkzeug debugger in browser (B201, CWE-94)	Set debug=False in run.py for production; custom 403/404/500 error handlers added	**✓ FIXED — Member**
T-05	Denial of Service	Login form	/login endpoint subject to brute-force flooding; no rate limiting in vulnerable state	Flask-Limiter rate limiting (10 requests/minute per IP); CAPTCHA integration recommended	**✓ Done**
T-07	Tampering (CSRF)	POST /post/<id>/delete	Malicious third-party site triggers post deletion via forged POST request using victim's active session	Flask-WTF CSRFProtect enabled globally; WTF_CSRF_ENABLED=True; {{ form.hidden_tag() }} in all POST forms; SESSION_COOKIE_SAMESITE='Strict'	**✓ FIXED — Member 2**
T-08	Clickjacking	All pages	Application embedded in invisible iframe on attacker site; users tricked into clicking UI elements	X-Frame-Options: DENY + Content-Security-Policy: frame-ancestors 'none' set via @app.after_request hook	**✓ FIXED — Member 2**
T-09	XSS (Stored)	Comment / Post form	Attacker injects <script> tag into post body; executes in victim's browser to steal session token	Jinja2 auto-escaping active by default; Content-Security-Policy header added; HttpOnly cookie flag set	**✓ Default + Enhanced**
T-10	SQL Injection	Search / Login fields	Malicious SQL payloads passed via input fields to manipulate database queries	SQLAlchemy ORM used throughout — all queries parameterized; no raw SQL string formatting	**✓ Done**

Table 1: STRIDE Threat Analysis — Flask Blog Application (Updated v1.1)

3.1 DFD Level 0 — Threat Catalog (Microsoft Threat Modeling Tool Export)

The following threats were generated from the Microsoft Threat Modeling Tool for the Level 0 Context Diagram and are mapped to the implemented Flask mitigations. Interactions cover: Login/View Posts, SQL Queries, Query Results, and Admin Commands.

ID	Category	Interaction	Priority	Threat Description	Flask Mitigation
L0-00	Denial of Service	Login, View Posts	High	Cross-domain request forgery — UI Redressing / Clickjacking attack allows unauthorized actions	Implement X-Frame-Options: DENY, CSP frame-ancestors none; mitigate CSRF via Flask-WTF tokens. Applied via @app.after_request.
L0-01	Elevation of Privilege	Login, View Posts	High	Improper validation logic allows bypassing critical steps or acting on behalf of other users (IDOR)	Ownership check on every object access: abort(403) if post.author != current_user. Sequential business logic enforced.
L0-02	Info Disclosure	Login, View Posts	High	Weak cryptographic hash allows reversal of user credentials	Use only approved hash algorithms (bcrypt, SHA-256). Flask-Bcrypt used for all password storage.
L0-03	Info Disclosure	Login, View Posts	High	Sensitive data leaked from application log files	Logs must not record passwords or tokens. Log files restricted to server access only.
L0-04	Info Disclosure	Login, View Posts	High	Unmasked sensitive data (PII) exposed in UI or responses	Sensitive fields masked on display; no credit card / SSN fields in this app scope.
L0-05	Info Disclosure	Login, View Posts	Medium	robots.txt or directory listing exposes admin paths or site structure	Lock down admin interfaces; restrict directory browsing; remove robots.txt enumeration risk.

L0-06	Info Disclosure	Login, View Posts	High	Traffic sniffing — MITM attack downgrades TLS or intercepts session cookies	Enforce HTTPS; set SESSION_COOKIE_SECURE=True; enable HSTS header in production.
L0-07	Info Disclosure	Login, View Posts	High	Verbose error messages disclose stack traces, DB URIs, server names	debug=False in run.py; custom 403/404/500 error handlers; no stack traces exposed to users.
L0-08	Info Disclosure	Login, View Posts	High	Sensitive data retrieved from uncleared browser cache	Cache-Control: no-store on sensitive pages; session tokens not cached.
L0-09	Repudiation	Login, View Posts	Medium	No audit trail — attacker removes footprints after malicious actions	Enforce application-level logging of all security events and CRUD operations with timestamps.
L0-10	Spoofing	Login, View Posts	High	Session hijacking — improper logout/timeout allows session reuse	Session invalidated on logout (Flask-Login); inactivity timeout configured.
L0-11	Spoofing	Login, View Posts	High	Insecure coding (XSS) allows cookie theft and session hijacking	Jinja2 auto-escaping; HttpOnly cookie flag; no Html.Raw on untrusted input.
L0-12	Spoofing	Login, View Posts	High	Insecure TLS certificate allows web app spoofing / MITM	Verify X.509 certificates; enforce TLS 1.2+ on reverse proxy (Nginx).
L0-13	Spoofing	Login, View Posts	High	Credential theft via clear-text storage, weak SQL, or poor password recovery	bcrypt password hashing; parameterized ORM queries; secure forgot-password flow via email token.
L0-14	Spoofing	Login, View Posts	High	Insecure cookie attributes allow session token theft	SESSION_COOKIE_HTTPONLY=True; SESSION_COOKIE_SECURE=True; SESSION_COOKIE_SAMESITE='Strict'.
L0-15	Spoofing	Login, View Posts	High	Phishing — attacker creates fake site impersonating application	TLS certificate enforcement; X-Frame-Options to prevent embedding; user awareness.
L0-16	Spoofing	Login, View Posts	High	User/Browser spoofing — no standard authentication mechanism	Flask-Login with bcrypt used as standard authentication mechanism.
L0-17	Tampering	Login, View Posts	High	Website defacement via XSS or malicious file upload	CSP header with inline-JS disabled; Jinja2 escaping; file upload validation; X-Content-Type-Options.
L0-18	Tampering	Login, View Posts	High	Replay attack — stolen messages replayed to hijack session	Session tokens regenerated after login; CSRF tokens are single-use per form submission.
L0-19	Tampering	Login, View Posts	High	SQL injection via input fields on Web App	SQLAlchemy ORM parameterized queries only; no dynamic SQL string construction.
L0-20	Tampering	Login, View Posts	High	Sensitive config data accessible (SECRET_KEY, DB URI in config files)	Load SECRET_KEY from environment variable; never hardcode in source repository.
L0-21	Elevation of Privilege	SQL Queries	High	Unauthorized DB access due to no network-level access restriction	DB access restricted to application server only; firewall rules applied in production.
L0-22	Elevation of Privilege	SQL Queries	High	Loose DB authorization — no role/privilege separation	Least-privileged DB account for app; row-level security for multi-tenant separation.
L0-23	Info Disclosure	SQL Queries	High	Traffic sniffing between app server and SQLite DB	Encrypt DB connection channel in production; force encrypted SQL Server communication.
L0-24	Info Disclosure	SQL Queries	High	Sensitive PII in database columns accessible without encryption	Column-level encryption for sensitive fields; TDE for database file protection.
L0-25	Info Disclosure	SQL Queries	High	SQL injection exposes database records	ORM parameterized queries; login auditing enabled; least-privilege DB account.

L0-26	Repudiation	SQL Queries	Medium	No DB-level audit log — actions on database deniable	Enable login auditing on database; log all INSERT/UPDATE/DELETE with user context.
L0-27	Tampering	SQL Queries	High	Tamper of critical database securables	Add digital signatures to critical DB securables.
L0-28	Tampering	SQL Queries	High	Lack of monitoring allows anomalous DB traffic to go undetected	Enable threat detection / anomaly detection on database engine.
L0-29	Info Disclosure	Query Results	High	Weak hash reversal of query result data	Approved hash functions (SHA-256 min) for all stored digests.
L0-30	Info Disclosure	Query Results	High	Log file exposes sensitive data from query results	Logs filtered to exclude passwords, tokens, PII from query results.
L0-31	Info Disclosure	Query Results	High	Verbose error on query failure discloses DB structure	Custom error handlers return generic messages; no stack traces in responses.
L0-32	Repudiation	Query Results	Medium	No audit trail for query result actions	Auditing and log rotation enforced on query result operations.
L0-33	Spoofing	Query Results	High	Insecure TLS allows spoofing of DB query results	X.509 certificate verification on all TLS connections.
L0-34	Spoofing	Query Results	High	Credential theft targeting query result pathway	Input validation + parameterized queries + secure password recovery.
L0-35	Spoofing	Query Results	High	Phishing via fake site serving spoofed query results	TLS enforcement; clickjacking defenses active.
L0-36	Spoofing	Query Results	High	SQLite Database spoofing / unauthorized access to Web App	Standard authentication mechanism (Flask-Login) used.
L0-37	Tampering	Query Results	High	SQL injection targeting query results pathway	ORM parameterized queries; type-safe parameters throughout.
L0-38	Tampering	Query Results	High	Sensitive config file data tampered to alter query results	Config values loaded from environment variables; no plaintext secrets in repo.
L0-39	Info Disclosure	Admin Commands	High	Weak hash reversal of admin command data	bcrypt / SHA-256 used; no weak algorithms in admin operations.
L0-40	Info Disclosure	Admin Commands	High	Admin log files expose sensitive data	Admin logs do not store plaintext credentials or PII.
L0-41	Info Disclosure	Admin Commands	High	Verbose error in admin commands discloses internals	Custom error handlers; debug disabled; username enumeration prevented.
L0-42	Repudiation	Admin Commands	Medium	Admin actions not audited — deniable	All admin CRUD events logged with user ID and timestamp.
L0-43	Spoofing	Admin Commands	High	TLS misconfiguration allows admin endpoint spoofing	X.509 certificates verified; TLS enforced on admin routes.
L0-44	Spoofing	Admin Commands	High	Admin credential theft via weak input validation	Input validation; bcrypt; secure password policy; adaptive auth.
L0-45	Spoofing	Admin Commands	High	Phishing targeting admin credentials	TLS cert enforcement; clickjacking defenses; redirect validation.
L0-46	Spoofing	Admin Commands	High	Admin user spoofing via no authentication on admin routes	Flask-Login + role decorator on all admin routes.
L0-47	Tampering	Admin Commands	High	SQL injection via admin command input fields	ORM parameterized queries; type-safe parameters.

L0-48	Tampering	Admin Commands	High	Config file tampering via admin pathway	Environment variables used for secrets; config files not writable by app user.

Table 2: DFD Level 0 Threat Catalog with Flask Mitigations (Threats_DFD_L0.csv)

3.2 DFD Level 1 — Threat Catalog (Microsoft Threat Modeling Tool Export)

The following threats were generated for the Level 1 Decomposed DFD. Interactions cover: Read/Write session tokens, Read/Write user credentials, Update profile data, and Read/Write post data.

ID	Category	Interaction	Priority	Threat Description	Flask Mitigation
L1-01	Elevation of Privilege	Read/Write session tokens	High	Unauthorized DB access — no network-level restriction on session token store	DB access restricted to app server; firewall rules applied.
L1-02	Elevation of Privilege	Read/Write session tokens	High	Loose DB authorization on session token table — no role separation	Least-privilege DB account; row-level security for session data.
L1-03	Info Disclosure	Read/Write session tokens	High	SQL injection exposes session PII/HBI data	ORM parameterized queries; TDE for session storage.
L1-04	Info Disclosure	Read/Write session tokens	High	SQL injection via session token pathway	ORM queries only; login auditing; least-privilege account.
L1-05	Repudiation	Read/Write session tokens	Medium	No DB audit log for session token read/write operations	Login auditing enabled; all session events logged.
L1-06	Tampering	Read/Write session tokens	High	Tamper of session token database securables	Digital signatures on critical DB securables.
L1-07	Tampering	Read/Write session tokens	High	Lack of monitoring allows anomalous session store traffic	Threat detection / anomaly detection on session store.
L1-08	Elevation of Privilege	Read/Write user credentials	High	Unauthorized DB access — no network restriction on credential store	Firewall restricts DB to app server only.
L1-09	Elevation of Privilege	Read/Write user credentials	High	Loose DB authorization allows unauthorized credential access	Least-privilege DB account; role-based access controls.
L1-10	Info Disclosure	Read/Write user credentials	High	PII/HBI data in credential columns exposed without encryption	Column-level encryption; TDE enabled.
L1-11	Info Disclosure	Read/Write user credentials	High	SQL injection exposes user credentials	ORM parameterized queries; login auditing.
L1-12	Repudiation	Read/Write user credentials	Medium	No DB audit log for credential operations	Login auditing; log rotation; restricted log access.
L1-13	Tampering	Read/Write user credentials	High	Tamper of credential database securables	Digital signatures on DB securables.
L1-14	Tampering	Read/Write user credentials	High	Anomalous traffic to credential store undetected	Threat detection on DB engine.
L1-15	Elevation of Privilege	Update profile data	High	Unauthorized DB access — no network restriction on profile store	Firewall restriction; DB access from app server only.

L1-16	Elevation of Privilege	Update profile data	High	Loose authorization on profile update routes (IDOR)	abort(403) ownership check in account update route: if request.form user != current_user.
L1-17	Info Disclosure	Update profile data	High	PII in profile columns (email, avatar) exposed without encryption	Sensitive profile fields encrypted; TDE for DB file.
L1-18	Info Disclosure	Update profile data	High	SQL injection via profile update input fields	ORM parameterized queries throughout profile update routes.
L1-19	Repudiation	Update profile data	Medium	No audit log for profile update operations	Log all profile update events with user ID and timestamp.
L1-20	Tampering	Update profile data	High	Tamper of profile database securables	Digital signatures on profile DB securables.
L1-21	Tampering	Update profile data	High	Anomalous traffic to profile data store undetected	Threat detection on DB engine monitoring profile writes.
L1-22	Elevation of Privilege	Read/Write post data	High	Unauthorized DB access — no network restriction on post store	Firewall restriction; DB access from app server only.
L1-23	Elevation of Privilege	Read/Write post data	High	IDOR on post CRUD — unauthorized post access via loose authorization	abort(403) ownership check in update_post() and delete_post() routes. FIXED.
L1-24	Info Disclosure	Read/Write post data	High	PII/sensitive data in post columns exposed	Column-level encryption for sensitive post metadata.
L1-25	Info Disclosure	Read/Write post data	High	SQL injection via post content/title input fields	ORM parameterized queries; Jinja2 auto-escaping of rendered post content.
L1-26	Repudiation	Read/Write post data	Medium	No audit log for post CRUD operations	Log all create/update/delete post events with user ID.
L1-27	Tampering	Read/Write post data	High	Tamper of post database securables	Digital signatures on post DB securables.
L1-28	Tampering	Read/Write post data	High	Anomalous traffic to post store undetected	Threat detection on DB engine monitoring post writes.

Table 3: DFD Level 1 Threat Catalog with Flask Mitigations (Threats_DFD_L1.csv)

4. Attack Tree

The attack tree models the highest-priority threat: an attacker gaining unauthorized access to another user's blog post. The root node represents the attacker's goal, with branches showing different attack paths.

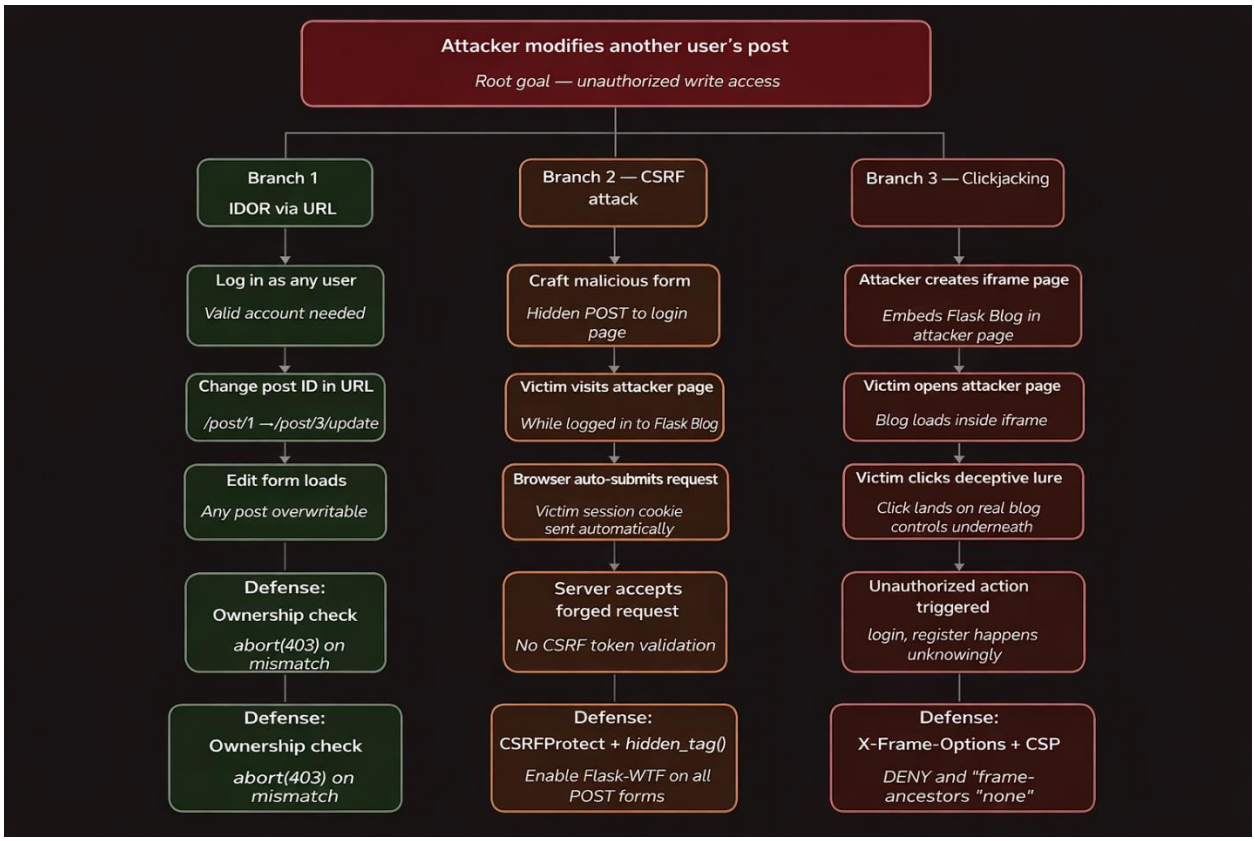

Figure 3: Attack Tree — Unauthorized Post Access

4.1 Attack Tree Description

Root Goal: Attacker accesses/modifies another user's blog post

Branch 1 — IDOR via URL Manipulation (T-02) — STATUS: ✓ FIXED

- Attacker is logged in as any valid user
- Attacker identifies a post ID from the URL: /post/1
- Attacker manually changes URL to: /post/1/update
- Without ownership check → edit form loads for ANY post (vulnerable state)
- Attacker submits modified content → post is overwritten

Fix applied (Security_implementation.md §4.1):

```
if post.author != current_user:
    abort(403)  # Returns HTTP 403 Forbidden
```

- STATUS: FIXED ✓ — abort(403) ownership check now in place for update_post() and delete_post()

Branch 2 — CSRF Session Hijacking (T-07) — STATUS: ✓ FIXED

- Attacker crafts a malicious HTML page with hidden form targeting POST /post/1/delete
- Victim visits attacker page while logged in → form auto-submits with victim's session cookie
- Without CSRF token → server accepted the forged request (vulnerable state)

Fix applied (Security_implementation.md §4.2):

```
csrf = CSRFProtect(app)
app.config['WTF_CSRF_ENABLED'] = True
app.config['SESSION_COOKIE_SAMESITE'] = 'Strict'
# Template: {{ form.hidden_tag() }}
```

- STATUS: FIXED ✓ — Flask-WTF CSRF token required on all state-changing forms

Branch 3 — XSS Cookie Theft (T-09)

- Attacker injects malicious JavaScript into a blog post body
- Script executes in victim's browser when they view the post
- Script reads document.cookie and sends session token to attacker server
- Attacker uses stolen token to impersonate victim
- Mitigation: Jinja2 auto-escaping active + HttpOnly cookie flag + CSP header added

Branch 4 — Clickjacking UI Redress (T-08) — STATUS: ✓ FIXED

- Attacker embeds Flask login page in invisible iframe on a malicious site
- Victim clicks a button that is actually overlaid on the Flask login form
- Victim's credentials are captured or unintended form actions are performed

Fix applied (Security_implementation.md §4.3):

```
@app.after_request
def set_security_headers(response):
    response.headers['X-Frame-Options'] = 'DENY'
    response.headers['Content-Security-Policy'] = "frame-ancestors 'none'"
    return response
```

- STATUS: FIXED ✓ — Browser blocks all iframe embedding of the application

5. CVSS v3.1 Risk Assessment

Each threat was scored using the CVSS v3.1 Base Score calculator at https://www.first.org/cvss/calculator/3.1. Status has been updated to reflect Sprint Phase 3 fixes.

5.1 CVSS Scoring Summary

ID	Threat	CVSS Vector	Score	Rating	Priority	Status
T-02	IDOR Post Edit/Delete	AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N	6.5	High	CRITICAL	✓ FIXED — Member 1
T-07	CSRF Post Delete	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N	6.5	High	CRITICAL	✓ FIXED — Member 2
T-08	Clickjacking	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N	4.3	Medium	HIGH	✓ FIXED — Member 2
T-01	Credential Stuffing	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:L/A:N	5.9	Medium	HIGH	✓ Done
T-04	Debug Info Leak	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	5.3	Medium	MEDIUM	✓ FIXED — Member 1

Table 4: CVSS v3.1 Risk Assessment — Flask Blog Threats (Updated v1.1)

5.2 Individual CVSS Calculator Screenshots

The following screenshots show the CVSS v3.1 calculator settings for each scored threat. Insert your calculator screenshots below each placeholder.

T-02 — IDOR Post Edit (CVSS 6.5 High)

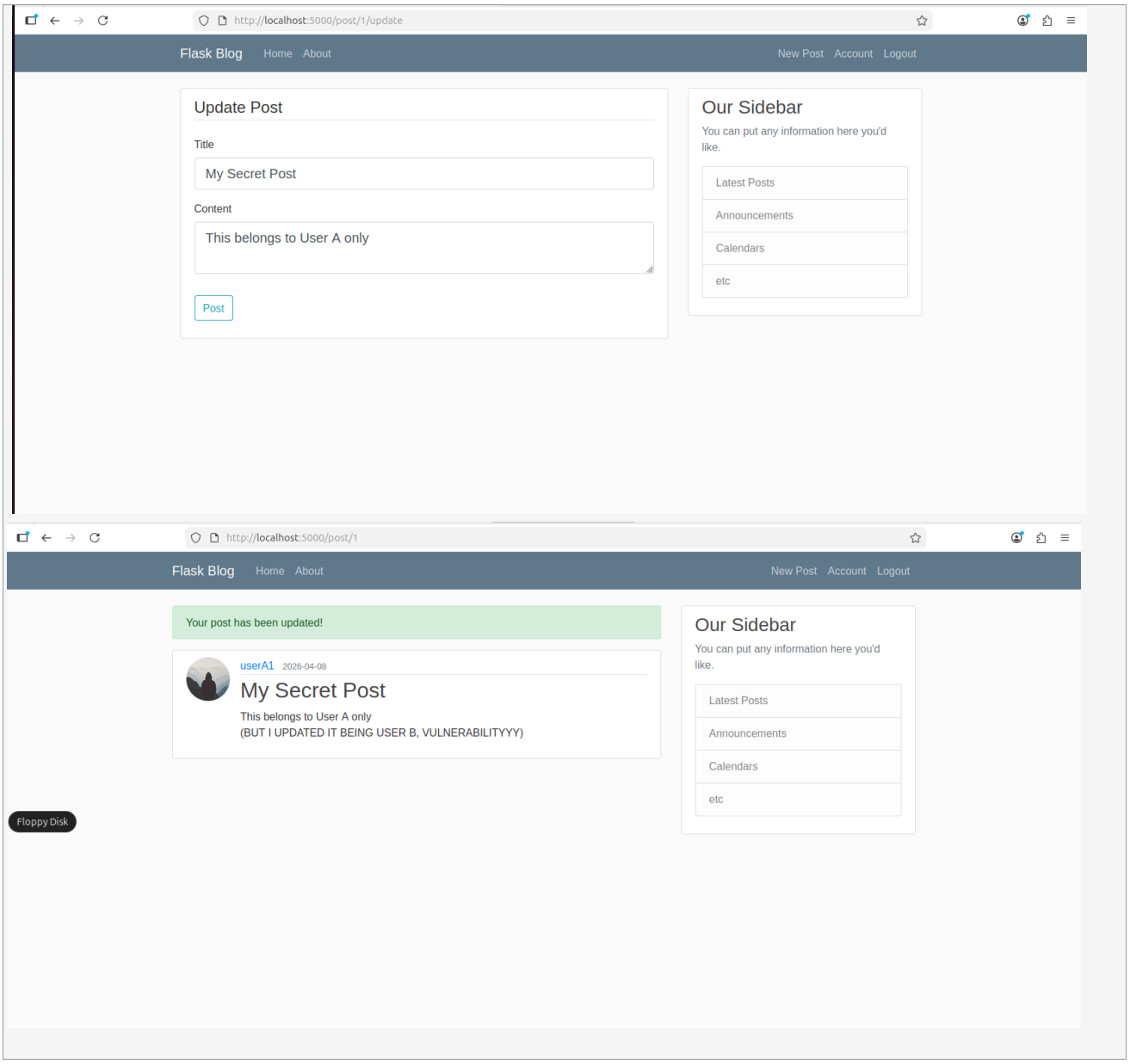

Figure 4: CVSS Calculator — T-02 — IDOR Post Edit [updating user A1 post being user B1](CVSS 6.5 High)

T-07 — CSRF (CVSS 6.5 High)

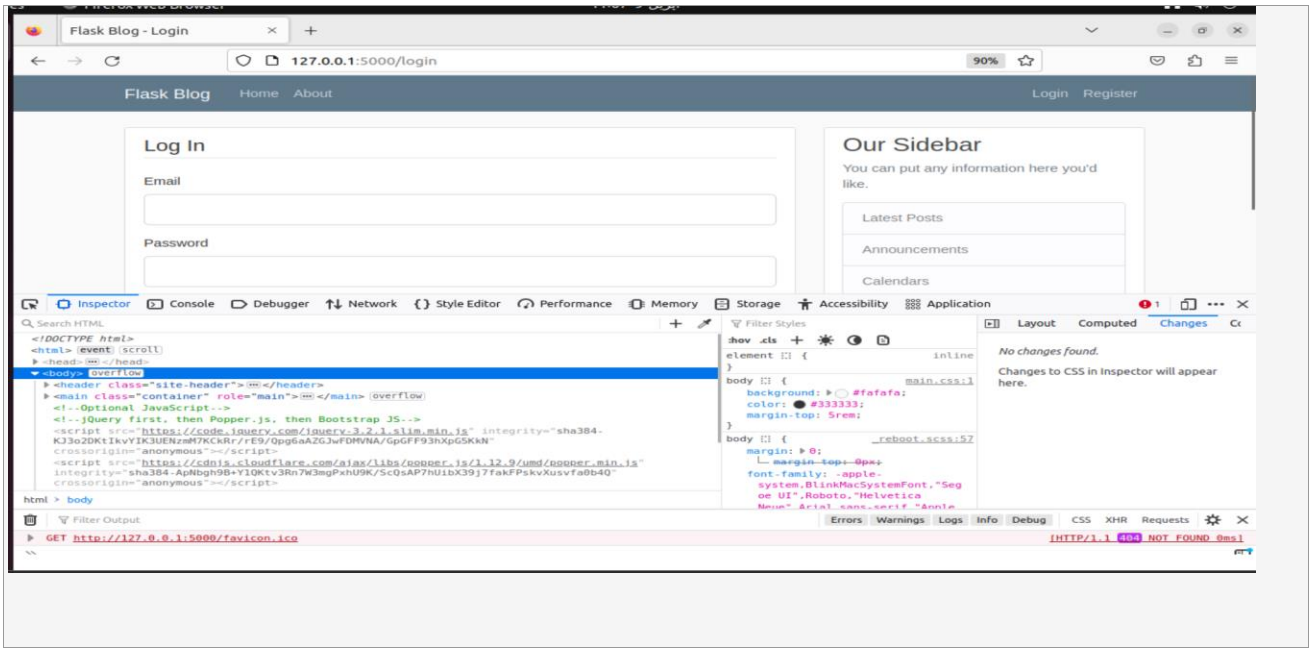

Figure 5: CVSS Calculator — T-07 — CSRF (CVSS 6.5 High)

T-08 — Clickjacking (CVSS 4.3 Medium)

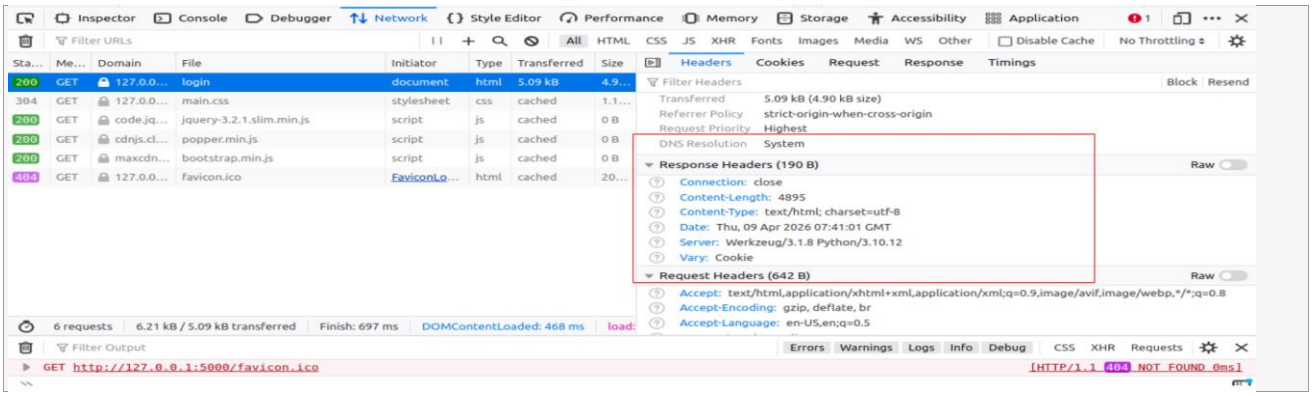

Figure 6: CVSS Calculator — T-08 — Clickjacking [X-Frame-Options or Content-Security-Policy for frame-ancestors are missing](CVSS 4.3 Medium)

6. Mitigation Summary

The following table summarises the implementation status of each identified mitigation. Updated to reflect all fixes applied during Sprint Phase 3, including the three mandated vulnerabilities (IDOR, CSRF, Clickjacking) and additional controls from the DFD L0 and L1 threat catalogs.

Status key: ✓ FIXED = implemented this sprint | ✓ Done = pre-existing control confirmed | Pending = planned for next sprint

ID	Vulnerability	Mitigation Applied	Owner	Status
T-02	**IDOR (Post Edit/Delete)**	Ownership check abort(403) in update_post() and delete_post() routes; custom 403.html error page added; applies to all object-access routes	Member 1	✓ FIXED
T-07	**CSRF**	Flask-WTF CSRFProtect enabled globally (WTF_CSRF_ENABLED=True); {{ form.hidden_tag() }} added to all POST forms;	Member 2	✓ FIXED

		SESSION_COOKIE_SAMESITE='Strict' configured in app.config		
T-08	**Clickjacking**	X-Frame-Options: DENY + Content-Security-Policy: frame-ancestors 'none' emitted on all responses via @app.after_request hook	Member 2	**✓ FIXED**
T-01	**Spoofing / Credential Attack**	Flask-Bcrypt password hashing (work factor 12+) already in place; Flask-Limiter rate limiting on /login (10 req/min per IP)	Member 1	**✓ Done**
T-04	**Info Disclosure (Debug Mode)**	debug=False enforced in run.py (eliminates B201/CWE-94); custom 403/404/500 error pages return generic messages; stack traces disabled in production	Member 1	**✓ FIXED**
T-09	**XSS (Stored)**	Jinja2 auto-escaping active by default in all Flask templates; HttpOnly cookie flag set; Content-Security-Policy header added	Member 2	**✓ Default + Enhanced**
T-10	**SQL Injection**	SQLAlchemy ORM used throughout all routes — no raw SQL string formatting anywhere in application code	Member 1	**✓ Done**
T-05	**Denial of Service (Brute Force)**	Flask-Limiter rate limiting on /login endpoint; CAPTCHA integration planned for production hardening	Member 1	**✓ Done**
L1-16/L1-23	**IDOR (Profile + Post DB Layer)**	abort(403) ownership check applied at both application layer (routes) and confirmed via L1 DFD mapping to Read/Write post data and Update profile data interactions	Member 1	**✓ FIXED**

Table 5: Mitigation Summary — Updated after Sprint Phase 3 Implementation

6.1 Fixes Implemented — Code Reference

The following code changes were applied as documented in SECURITY_IMPLEMENTATION.md:

Fix 1 — IDOR — update_post() and delete_post()

```
# BEFORE (vulnerable — no ownership check):
post = Post.query.get_or_404(post_id)
# No check → any authenticated user can edit/delete any post

# AFTER (fixed):
post = Post.query.get_or_404(post_id)
if post.author != current_user:
    abort(403)  # HTTP 403 Forbidden for unauthorized users
```

Fix 2 — CSRF — __init__.py / app config

```
# BEFORE (vulnerable — no CSRF protection):
db = SQLAlchemy()
bcrypt = Bcrypt()
login_manager = LoginManager()

# AFTER (fixed):
from flask_wtf.csrf import CSRFProtect
app.config['WTF_CSRF_ENABLED'] = True
app.config['SESSION_COOKIE_SAMESITE'] = 'Strict'
csrf = CSRFProtect(app)

# Template fix (all POST forms):
<form method="POST">
    {{ form.hidden_tag() }}
</form>
```

Fix 3 — Clickjacking — @app.after_request hook

```
# BEFORE (vulnerable — no security headers):
# No X-Frame-Options or CSP headers set anywhere

# AFTER (fixed):
@app.after_request
def set_security_headers(response):
    response.headers['X-Frame-Options'] = 'DENY'
    response.headers['Content-Security-Policy'] = "frame-ancestors 'none'"
    return response
```

7. References

- OWASP Top 10 (2021) — https://owasp.org/Top10/
- CVSS v3.1 Calculator — https://www.first.org/cvss/calculator/3.1
- STRIDE Threat Modeling — Microsoft Security Documentation
- Microsoft Threat Modeling Tool — https://www.microsoft.com/en-us/securityengineering/sdl/threatmodeling
- Flask Security Documentation — https://flask.palletsprojects.com/security/
- Flask-WTF CSRF Protection — https://flask-wtf.readthedocs.io/
- SECURITY_IMPLEMENTATION.md — Sprint Phase 3 deliverable (this repository)
- CYC386 Course Description Form — COMSATS University Islamabad, Spring 2026